tf.compat.v1.losses.softmax_cross_entropy Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/losses/softmax_cross_entropy

> *Creates a cross-entropy loss using tf.nn.softmax_cross_entropy_with_logits_v2.*

Function Signatures

```
tf.compat.v1.losses.softmax_cross_entropy( onehot_labels,
logits, weights=1.0, label_smoothing=0, scope=None,
loss_collection=ops.GraphKeys.LOSSES,
reduction=Reduction.SUM_BY_NONZERO_WEIGHTS )
```

Code Examples

```
tf.compat.v1.losses.softmax_cross_entropy(
onehot_labels,
logits,
weights=1.0,
label_smoothing=0,
scope=None,
loss_collection=ops.GraphKeys.LOSSES,
reduction=Reduction.SUM_BY_NONZERO_WEIGHTS
)
```

```
tf.compat.v1.losses.softmax_cross_entropy(
onehot_labels,
logits,
weights=1.0,
label_smoothing=0,
scope=None,
loss_collection=ops.GraphKeys.LOSSES,
reduction=Reduction.SUM_BY_NONZERO_WEIGHTS
)
```

```
loss = tf.compat.v1.losses.softmax_cross_entropy(
onehot_labels=onehot_labels,
logits=logits,
weights=weights,
label_smoothing=smoothing)
```

```
loss = tf.compat.v1.losses.softmax_cross_entropy(
onehot_labels=onehot_labels,
logits=logits,
weights=weights,
label_smoothing=smoothing)
```

```
loss_fn = tf.keras.losses.CategoricalCrossentropy(
from_logits=True,
label_smoothing=smoothing)
loss = loss_fn(
y_true=onehot_labels,
y_pred=logits,
sample_weight=weights)
```

```
loss_fn = tf.keras.losses.CategoricalCrossentropy(
from_logits=True,
label_smoothing=smoothing)
loss = loss_fn(
y_true=onehot_labels,
y_pred=logits,
sample_weight=weights)
```

```
y_true = [[0, 1, 0], [0, 0, 1]]
y_pred = [[0.05, 0.95, 0], [0.1, 0.8, 0.1]]
weights = [0.3, 0.7]
smoothing = 0.2
tf.compat.v1.losses.softmax_cross_entropy(y_true, y_pred,
weights=weights,
label_smoothing=smoothing).numpy()
0.57618
```

```
y_true = [[0, 1, 0], [0, 0, 1]]
```

Documentation

Creates a cross-entropy loss using tf.nn.softmax_cross_entropy_with_logits_v2.

Migrate to TF2

tf.compat.v1.losses.softmax_cross_entropy is mostly compatible with eager execution and tf.function. But, the loss_collection argument is ignored when executing eagerly and no loss will be written to the loss collections. You will need to either hold on to the return value manually or rely on tf.keras.Model loss tracking.

To switch to native TF2 style, instantiate the tf.keras.losses.CategoricalCrossentropy class with from_logits set as True and call the object instead.

Structural Mapping to Native TF2

Before:

After:

How to Map Arguments

Before & After Usage Example

Before:

After:

Description

Used in the notebooks

- Debug a TensorFlow 2 migrated training pipeline

weights acts as a coefficient for the loss. If a scalar is provided, then the loss is simply scaled by the given value. If weights is a tensor of shape [batch_size], then the loss weights apply to each corresponding sample.

If label_smoothing is nonzero, smooth the labels towards 1/num_classes: new_onehot_labels = onehot_labels * (1 - label_smoothing)

Note that onehot_labels and logits must have the same shape, e.g. [batch_size, num_classes]. The shape of weights must be broadcastable to loss, whose shape is decided by the shape of logits. In case the shape of logits is [batch_size, num_classes], loss is a Tensor of shape [batch_size].

Returns

Raises